ALGORITMOS E ESTRUTURAS DE DADOS

Métodos Internos de Ordenação de Dados

Aula 08 – A Evolução da Eficiência: Shell Sort, Quick Sort e Merge Sort



Shell Sort: O Poder dos Saltos



O Conceito

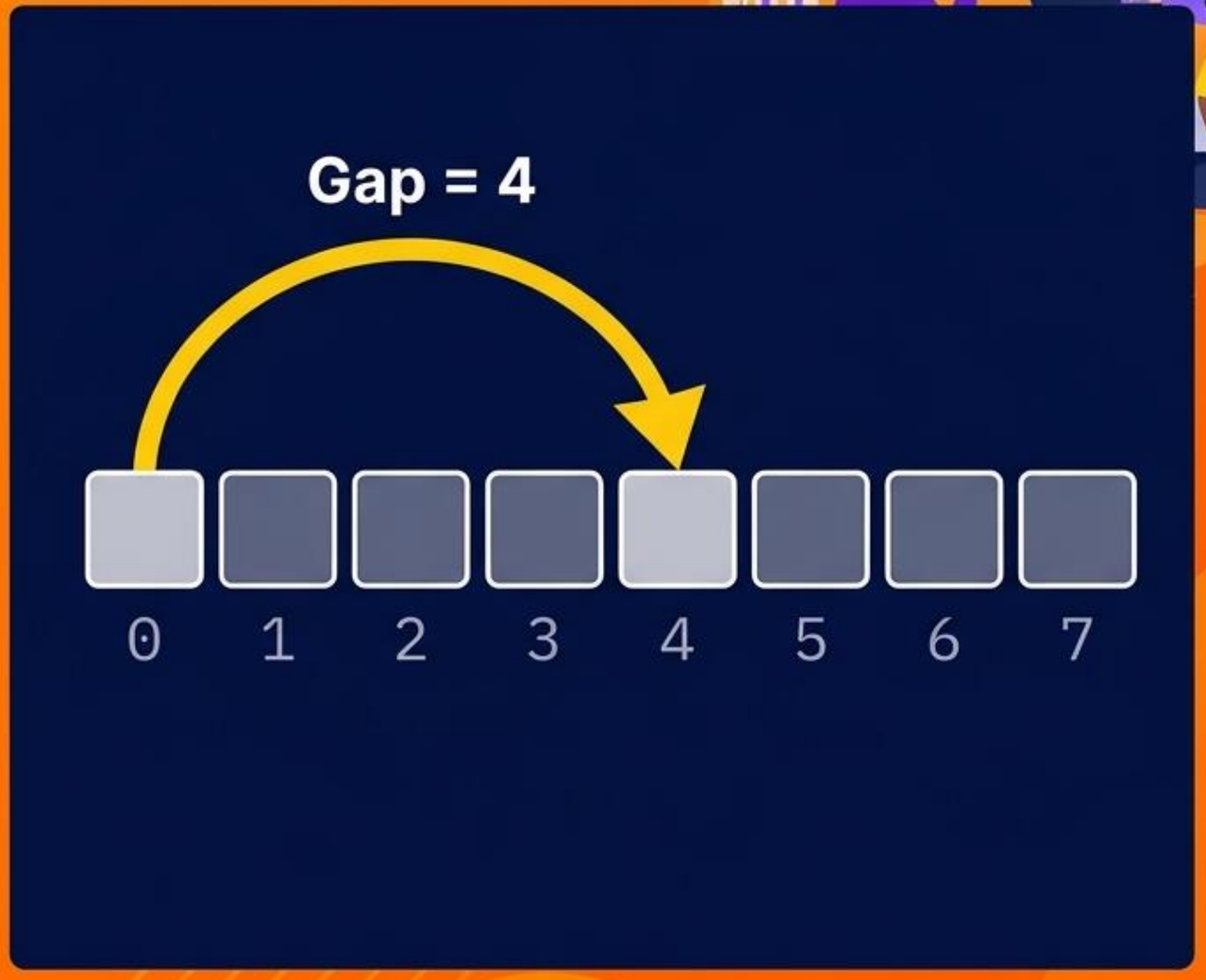
Uma extensão otimizada do Insertion Sort. Em vez de mover elementos casa por casa, o Shell Sort permite a troca de elementos distantes.

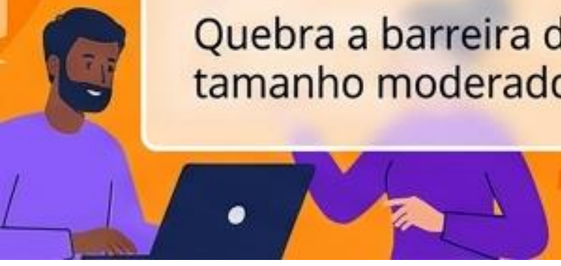
A Mecânica

Utiliza uma sequência de gaps (intervalos). O algoritmo divide a lista em sublistas virtuais espaçadas por esse gap. O gap diminui a cada iteração até chegar a 1.

Performance

Quebra a barreira do $O(n^2)$ para listas de tamanho moderado.





Shell Sort: Implementação C++



```cpp
#include <iostream>

void shellSort(int arr[], int n) {
    // Reduzindo o gap pela metade a cada iteração
    for (int gap = n / 2; gap > 0; gap /= 2) {

        // Fazendo um Insertion Sort para este gap
        for (int i = gap; i < n; i += 1) {
            int temp = arr[i];
            int j;

            // Deslocando elementos ordenados no gap atual
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
    }
}
```



Shell Sort: Rastreio Passo a Passo (Gap = 3)



Tabela 1 - Array Inicial

Valores	9	5	2	7	3	8
Índices	0	1	2	3	4	5

Com Gap = 3, comparamos e trocamos pares distantes. O 9 troca com o 7. O 5 troca com o 3. O 2 e o 8 já estão na ordem certa.

Tabela 2 - Após Gap=3

Valores	7	3	2	9	5	8
Índices	0	1	2	3	4	5



Shell Sort: A Varredura Final (Gap = 1)

Quando o gap chega a 1, o algoritmo funciona exatamente como um Insertion Sort padrão. A mágica? O vetor já está 'quase ordenado', exigindo pouquíssimas trocas.

Tabela 3 - Início Gap=1

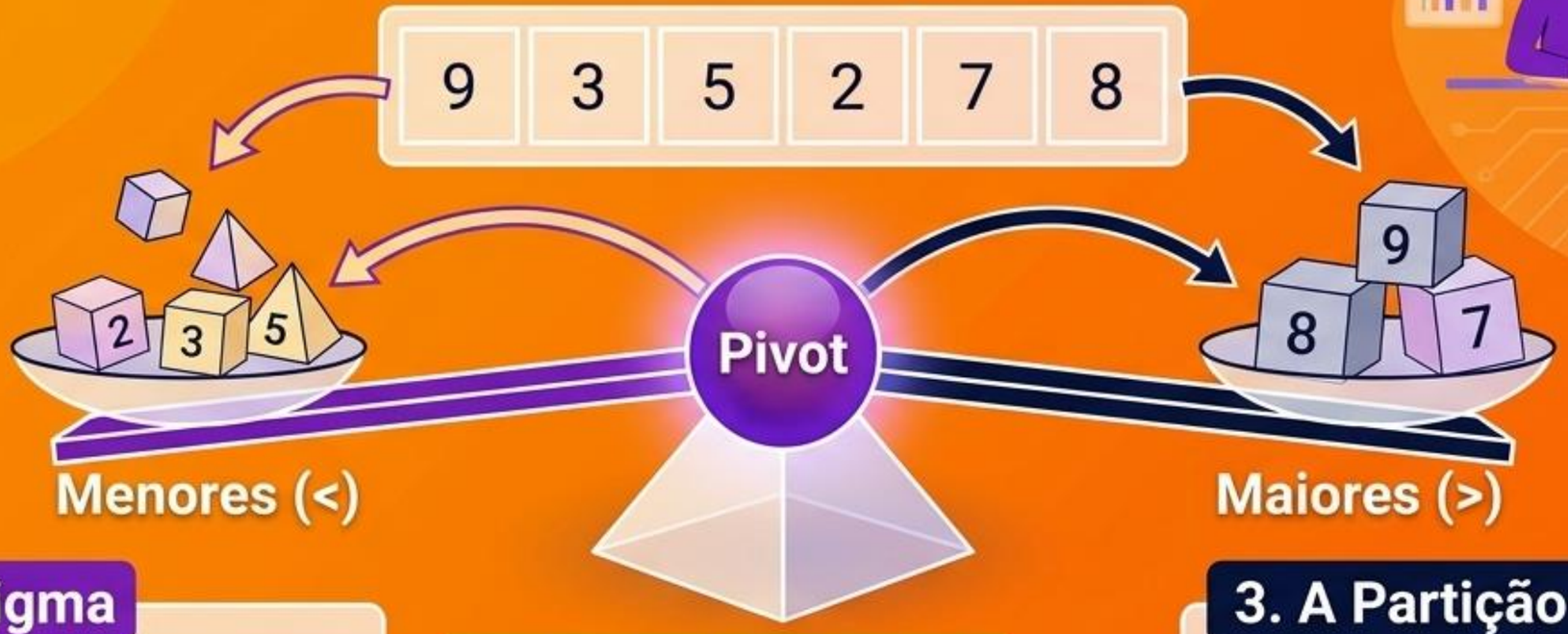


Tabela 4 - Array Ordenado

2	3	5	7	8	9

Quick Sort: Dividir para Conquistar





1. O Paradigma

Abandona a varredura linear e foca em quebrar o problema em partes menores **(recursividade)**.

2. A Escolha do Pivô

Um elemento é eleito como a 'âncora' da rodada (geralmente o último elemento do trecho).

3. A Partição

O núcleo do método. Reorganiza o vetor para que tudo **menor que o pivô fique à esquerda, e tudo maior fique à direita.**



Complexidade média impressionante de O(n log n).

Quick Sort: Implementação C++

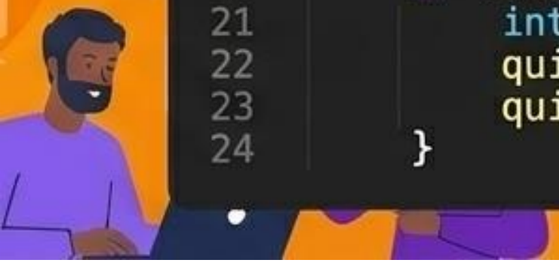


```cpp
#include <iostream>

// O coração do algoritmo: posicionar o pivô
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Escolhemos o último como pivô
    int i = (low - 1);     // Índice do menor elemento

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return (i + 1); // Retorna a posição final do pivô
}

// Função recursiva principal
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
```



NotebookLM

Quick Sort: A Partição na Prática



Tabela 1 - Análise Inicial

Ponteiro j

PIVÔ

8	3	1	7	0	10	2

Ponteiro i

O ponteiro j varre o array buscando valores menores que o pivô (2). Ao encontrar (como o 1 e o 0), o ponteiro i avança e os valores são trocados.

Tabela 2 - Após Trocas do Loop

1	0	8	7	3	10	2



Quick Sort: O Pivô Encontra Seu Lar



O loop termina. O pivô (2) é trocado com o primeiro elemento maior que ele (8).

Tabela 3 - Posicionando o Pivô

1	0	2	7	3	10	8

Esquerda (Menores que 2)

1	0

Direita (Maiores que 2)

7	3	10	8



O pivô está na sua posição final e definitiva. Agora, a função quickSort é chamada recursivamente para a metade esquerda e direita.

Merge Sort: A Conquista Garantida



1. Divisão Extrema:

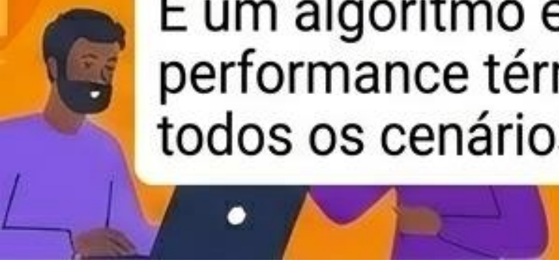
Corta o vetor rigorosamente ao meio, repetidas vezes, até que cada sublista tenha apenas 1 elemento (um elemento único já está, por definição, ordenado).

2. A Intercalação (Merge):

O grande segredo. Combina meticulosamente essas sublistas minúsculas em listas maiores, mantendo a ordem a cada passo.

3. A Grande Vantagem:

É um algoritmo estável e garante a melhor performance térmica possível de **O(n log n)** em todos os cenários (pior, médio e melhor caso).



Merge Sort: Implementação C++



```cpp
#include <iostream>

// Função auxiliar merge() junta dois subarrays ordenados
// (Lógica de intercalação omitida por espaço)
void merge(int arr[], int left, int mid, int right);

// Função recursiva de divisão
void mergeSort(int arr[], int left, int right) {
    if (left >= right)
        return; // Condição de parada: sublista de 1 elemento

    int mid = left + (right - left) / 2;

    // Divide e ordena a primeira e a segunda metade
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);

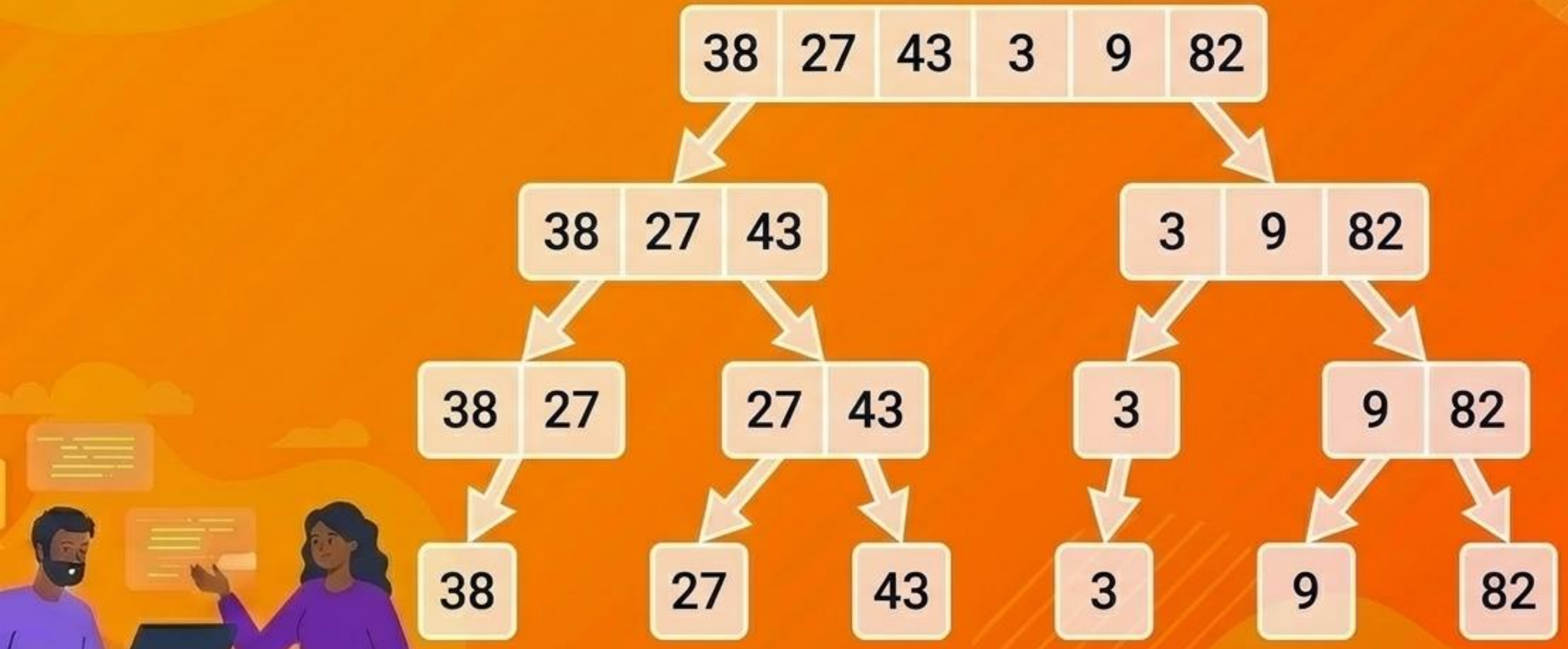
    // Junta (Merge) as duas metades agora ordenadas
    merge(arr, left, mid, right);
}
```



A mágica real estrutural acontece na função **merge()**, que necessita de um vetor auxiliar para intercalar ordenadamente as duas metades.

Merge Sort: A Fase de Divisão

O algoritmo entra em recursão profunda, fatiando os dados sem fazer nenhuma comparação, até chegar à atomicidade.



Merge Sort: A Fase de Intercalação (Merge)

Na volta da recursão, as sublistas são combinadas. Elementos das duas listas são comparados um a um, fluindo para uma nova lista na ordem exata.





Exercícios de Fixação



Shell Sort

1. Ordene o array `[14, 33, 27, 10, 35, 19, 42]` usando gaps de 3 e depois 1. Apresente as tabelas de estado após cada gap.

2. Aplique Shell Sort no vetor `[50, 12, 65, 30, 22, 9]` indicando graficamente as setas de troca.

Quick Sort

1. Ordene o vetor `[24, 9, 29, 14, 19, 27]` usando o último elemento (27) como pivô. Desenhe a tabela mostrando os ponteiros i e j até a primeira partição.
2. Encontre a posição correta do pivô '15' no array `[18, 12, 5, 20, 15, 8, 15]` e mostre as duas sublistas resultantes.







Exercícios de Fixação



Merge Sort

1. Desenhe a árvore completa de divisão e a árvore de mescla (merge) para o array `[11, 4, 7, 2, 9, 5]`.

2. Dadas as sublistas já ordenadas `[2, 8, 15]` e `[4, 5, 9]`, demonstre o processo de intercalação passo a passo, mostrando qual elemento é selecionado a cada rodada.



Pessoas impulsionando inovação.
Inovação impulsionando negócios.



C.E.S.A.R